Generatywne Sieci Przeciwstawne do generowania obrazów owoców i warzyw

Zuzanna Szewczyk

16 czerwca 2025

1 Wprowadzenie

Mój raport zacznę od zastosowania się do wskazówek z prezentacji z piątku. Pozbyłam się całkowicie problematycznego zbioru danych z pojazdami i zwierzętami, zastępując go zbiorem owoców i warzyw. Co znacząco wpłynęło na każdą kolejną próbę refaktoryzacji mojego kodu.

2 Dane

Użyłam zdjęć owoców i warzyw (głównie trenowałam na trzech różnych, to jest: bananach, kalafiorach i pomidorach). Wybierałam owoce/warzywa, które się mocno różnią, oraz pozostawałam przy trzech głównych, aby obliczenia nie trwały po parę godzin.

Dodatkowo, wszystkie obrazki miały rozmiar 100x100 pikseli, aby mało ważyły i nie zapchały mi całego miejsca na koncie studenckim. Nie posiadałam też jakoś bardzo dużo wersji tych warzyw/owoców — 77 bananów, 347 pomidorów i 234 kalafiorów.

3 Przykładowe dane

Poniżej przedstawiam przykładowe obrazy z każdej kategorii (po 3 z każdej):

Banany

Kalafiory

Pomidory

4 Dlaczego GANy, dalczego PyTorch czyli trochę moich problemów

Zanim przejdę do omawiania, czym jest w ogóle GAN, myślę, że powinnam wyjaśnić, skąd wziął się mój temat projektu. Na kursie z sztucznej inteligencji ostatnim zadaniem było właśnie zaimplementowanie cwgana. Jednak szybko się okazało, że to zwykłe zadanie na satori przerosło mnie trochę. W szczególności okazało się, że z do teraz niejasnych dla mnie powodów TensorFlow odmawiał współpracy z moją kartą graficzną lub kartą na miraclu. Stąd musiałam po raz pierwszy przerzucić się na PyTorch. Stąd też PyTorch pozostał w tym projekcie, bo dalej nie wiem czemu TensorFlow nie czytał mojej karty graficznej, a bez obliczeń na karcie graficznej moje programy działały zdecydowanie zbyt wolno. Również z tego powodu zmuszona byłam używać PyTorch i nie zastępowałam go na numPy czy jakąś inną bibliotekę (jedyną bardziej od podstaw, jaką znalazłam, która byłaby w stanie używać mojej karty, był JAX). Jednak ta biblioteka mnie przerosła, jak i problemy z nią i z kartą. Ze względu na te problemy serdecznie znienawidziłam Nvidię i wszystko, co z nią związane. Moje wcześniejsze rozwiązanie było bardzo naiwne i słabe. Radziło sobie na danych bez etykiet i bardzo mało skomplikowanych. Natomiast nawet na MNIST osiągał średnie efekty, bo szybko zaczynał rozmijać się z rzeczywistymi obrazami. Dlatego też postanowiłam go poprawić i to na lepszym zbiorze czyli owocach i warzywach.

5 Czym jest GAN

Generatywna Sieć Przeciwstawna (ang. Generative Adversarial Network, GAN) to para sieci neuronowych, które uczą się wspólnie poprzez grę minmaksową. Celem jest wygenerowanie nowych danych, które będą nieodróżnialne od danych rzeczywistych, w tym przypadku owoców i warzyw.

Dwie sieci: Generator i Dyskryminator

- **Generator (G)** — tworzy nowe obrazy na podstawie losowego szumu.
- **Dyskryminator (D)** — ocenia, czy obraz jest prawdziwy (z danych), czy sztuczny (z generatora).

Jak to działa?

1. Losujemy wektor szumu $z \sim \mathcal{N}(0, I)$.
2. Generator $G(z)$ przekształca szum w obraz x^*.
3. Dyskryminator $D(x)$ próbuje odróżnić *prawdziwe* obrazy $x \sim p_{data}$ od *wygenerowanych* $x^* = G(z)$.
4. Obie sieci uczą się na zmianę: Generator próbuje oszukać Dyskryminatora, a Dyskryminator próbuje się nie dać oszukać.

Równanie GAN: gra minimaksowa

$$\min_G \max_D \; \mathbb{E}_{x \sim p_{data}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))] \tag{1}$$

Oznaczenia:

- z — losowy szum wejściowy, np. 100 liczb z rozkładu normalnego.
- $G(z)$ — obraz wygenerowany przez Generator.
- $x \sim p_{data}$ — obraz z rzeczywistego zbioru danych (np. zdjęcie kalafiora).
- $D(x)$ — ocena Dyskryminatora, czy obraz wygląda na prawdziwy (liczba z przedziału $[0, 1]$), czym wyższa warotść tym bardziej jest pewny, że obrazek jest prawdziwy.

W skrócie:

- Dyskryminator D chce, aby $D(x) \to 1$ dla prawdziwych i $D(G(z)) \to 0$ dla fałszywych obrazów.
- Generator G chce, by $D(G(z)) \to 1$ — czyli aby oszukać dyskryminator
- Dyskryminator D chce **zmaksymalizować** $\log D(x) + \log(1 - D(G(z)))$.

- Generator G chce **zminimalizować** $\log(1 - D(G(z)))$.

W ten sposób obie sieci uczą się wspólnie. Generator poprawia jakość obrazów, a Dyskryminator staje się coraz lepszy w ocenie.

6 Czym jest CGAN

CGAN (Conditional GAN) to rozszerzenie klasycznego GAN-a, które pozwala dodanie etykiet.

Na czym polega różnica?

Zamiast generować losowy obraz, Generator dostaje dodatkową informację - etykietę klasy. Również Dyskryminator wie, jakiej klasy ma dotyczyć obraz, który ocenia.

Zmodyfikowane równanie CGAN:

$$\min_G \max_D \ \mathbb{E}_{x,y\sim p_{data}}[\log D(x,y)] + \mathbb{E}_{z\sim p_z, y\sim p_y}[\log(1 - D(G(z,y),y))] \tag{2}$$

Co się zmienia:

- Dodajemy y — etykietę klasy (np. "banan").
- Zarówno Generator jak i Dyskryminator dostają y jako dodatkowy argument.
- Generator $G(z,y)$ uczy się tworzyć obraz pasujący do y.
- Dyskryminator $D(x,y)$ uczy się rozpoznawać, czy obraz x pasuje do etykiety y.

7 Optymalizatory

Jak aktualizowałam wagi poprzez optymalizatory. Pierwsze próby okazały się niestabilne — używałam czystych gradientów, które szybko eksplodowały. Dlatego konieczne było wprowadzenie jakiejś optymalizacji. Zdecydowałam się na optymalizator Adam, który pozwolił na bardziej stabilny trening i uzyskanie pierwszych sensownych wyników.

1. Gradient

Najprostszy wariant optymalizacji polega na użyciu pełnego gradientu dla całego zbioru danych:

$$\theta \leftarrow \theta - \eta \cdot \nabla_\theta \mathcal{L}(\theta) \tag{3}$$

Gdzie:

- θ — parametry modelu (np. wagi sieci),
- η — współczynnik uczenia (learning rate),

- $\nabla_\theta \mathcal{L}(\theta)$ — gradient funkcji straty względem wag.

Wadą tej metody jest konieczność obliczenia gradientu na całym zbiorze danych, co bywa kosztowne.

3. Adam (Adaptive Momentum Estimation)

Adam to bardziej zaawansowany algorytm, który zamiast korzystać tylko z aktualnego gradientu, uwzględnia również historię wartości gradientów oraz ich kwadratów. Dzięki temu potrafi dynamicznie dostosować szybkość uczenia się do różnych parametrów modelu.

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t \qquad \text{(średnia z przeszłych gradientów)} \tag{4}$$

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2 \qquad \text{(średnia z przeszłych kwadratów gradientów)} \tag{5}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \qquad \text{(poprawka na początkowe wartości)} \tag{6}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{7}$$

$$\theta_t = \theta_{t-1} - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \tag{8}$$

Gdzie:

- g_t — gradient w czasie t,
- m_t — średnia z wcześniejszych gradientów (uśrednienie kierunku zmian),
- v_t — średnia z wcześniejszych kwadratów gradientów (szacowanie zmienności),
- η — współczynnik uczenia,
- ϵ — mała liczba zapobiegająca dzieleniu przez 0.

Zalety:

- automatyczna regulacja kroku uczenia,
- dobrze radzi sobie z niestabilnymi i rzadkimi gradientami (np. obrazy, tekst),
- szybka zbieżność nawet w trudnych zadaniach,
- bardzo dobrze działa w GAN-ach (z odpowiednio dobranymi parametrami).

3. RMSProp

RMSProp to algorytm, który zamiast korzystać z pełnej historii gradientów, utrzymuje jedynie uśrednioną wartość kwadratu gradientu z kilku ostatnich kroków. Działa podobnie do Adama, ale bez uwzględniania kierunku gradientów w czasie (czyli nie używa „średniej z przeszłych gradientów”).

$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot g_t^2 \tag{9}$$

$$\theta_t = \theta_{t-1} - \eta \cdot \frac{g_t}{\sqrt{v_t} + \epsilon} \tag{10}$$

Opis składników:

- g_t — gradient w czasie t,
- v_t — uśredniona wartość kwadratu gradientu,
- η — współczynnik uczenia,
- β — współczynnik wygładzania (typowo 0,9),
- ϵ — mała liczba zapobiegająca dzieleniu przez zero.

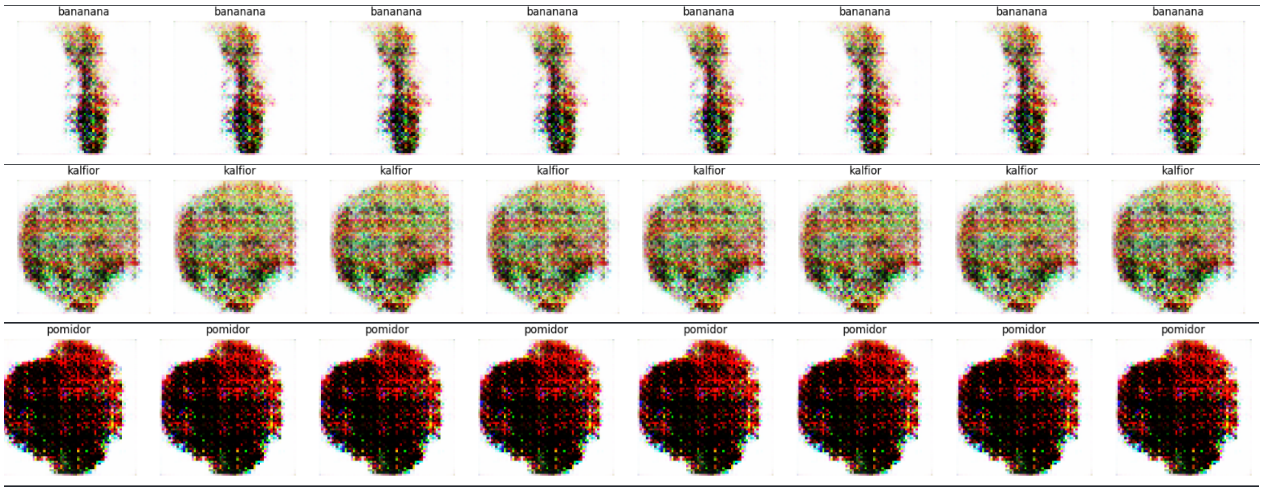
Jeśli dla jakiegoś parametru gradienty są bardzo duże, to v_t również będzie duże, co zmniejszy krok aktualizacji. Jeśli gradienty są małe, to krok będzie większy. Dzięki temu RMSProp potrafi lepiej radzić sobie z nierówną skalą danych i gradientów.

8 Moje wyniki

Początkowo zaczęłam od najprostszego kodu, czyli CGAN-a, który miał za zadanie generować obrazki na podstawie etykiet owoców i warzyw. Jak widać nie były to zbyt dobre wyniki, jednak byłam już w stanie zobaczyć jakieś zarysy i czasami kolory się zgadzały.

Wszystko było trenowane na losowo wymyślonych przeze mnie parametrach, bo na tym etapie projektu bardziej zależało mi na uruchomieniu samego modelu niż jego optymalizacji. Każdy model był trenowany przez 150 epok.

Poniżej przedstawiam przykładowe obrazy wygenerowane przez CGAN:



Obrazy wygenerowane przez CGAN: banan, kalafior, pomidor

9 Czym jest WGAN i CWGAN

Klasyczne GAN-y działają dobrze, ale mogą mieć problemy ze stabilnością treningu. Gdy Dyskryminator jest zbyt dobry, gradienty Generatora zanikają, co uniemożliwia dalsze uczenie.

Wasserstein GAN (WGAN)

WGAN to wariant GAN-a, który zamiast klasycznej funkcji kosztu opartej na entropii krzyżowej, wykorzystuje tzw. *odległość Wassersteina*.

Intuicja: Odległość Wassersteina mierzy „ile pracy trzeba wykonać, by przekształcić jedną rozkład masy w inny".

Poprzednia funkcja celu (klasyczny GAN): W klasycznym GAN-ie używano funkcji:

$$\min_G \max_D \ \mathbb{E}_{x\sim p_{data}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))] \tag{11}$$

Ta funkcja przestaje działać dobrze, gdy rozkłady danych i generatora nie pokrywają się — gradienty mogą wtedy znikać.

Nowa funkcja celu w WGAN

W WGANie zamiast $D(x) \in [0, 1]$, używa się **krytyka**, który zwraca wartość rzeczywistą, a nie z przedziału $[0, 1]$.

Funkcja celu krytyka:

$$\max_D \ \mathbb{E}_{x\sim p_{data}}[D(x)] - \mathbb{E}_{z\sim p_z}[D(G(z))] \tag{12}$$

Funkcja celu generatora:

$$\min_G \ \mathbb{E}_{z\sim p_z}[D(G(z))] \tag{13}$$

Warunek Lipschitza: Aby odległość Wassersteina była dobrze zdefiniowana i aproksymowalna, funkcja krytyka musi być lipschitzowska, czyli spełniać nierówność:

$$|D(x_1) - D(x_2)| \leq L \cdot \|x_1 - x_2\| \tag{14}$$

Dla nas: $L = 1$. Oznacza to, że funkcja nie może zmieniać się zbyt gwałtownie — musi być "gładka". W przeciwnym razie odległość Wassersteina traci sens jako miara.

Gradient penalty (GP): W praktyce zamiast przycinania wag, stosuje się karę. Dodaje się do funkcji kosztu dodatkowy składnik:

$$\text{GP} = \lambda(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2 \tag{15}$$

Gdzie $\hat{x}$ to punkt interpolowany między prawdziwym a fałszywym przykładem. Kara ta wymusza, aby normy gradientów krytyka względem danych wejściowych były bliskie 1 — co w praktyce oznacza, że funkcja D zachowuje się jak funkcja lipschitzowska.

Funkcja celu krytyka z GD:

$$\max_D \ \mathbb{E}_{x \sim p_{data}}[D(x)] - \mathbb{E}_{z \sim p_z}[D(G(z))] + \lambda \cdot GP \tag{16}$$

Funkcja celu generatora nie ulega zmianie:

$$\min_G \ \mathbb{E}_{z \sim p_z}[D(G(z))] \tag{17}$$

Zalety WGAN:

- Stabilniejszy trening,
- Gradienty nie zanikają,
- Funkcja kosztu dobrze koreluje z jakością generowanych obrazów,

CWGAN — WGAN warunkowy

Tak jak CGAN rozszerza GAN, CWGAN rozszerza WGAN o możliwość generowania z warunkiem (etykietą).

Funkcja celu CWGAN

W CWGAN krytyk i generator znają etykietę y. Funkcja celu wygląda tak:

Dla krytyka, o ile używamy GP:

$$\max_D \ \mathbb{E}_{x,y \sim p_{data}}[D(x, y)] - \mathbb{E}_{z \sim p_z, y \sim p_y}[D(G(z, y), y)] \tag{18}$$

Dla generatora:

$$\min_G \ -\mathbb{E}_{z \sim p_z, y \sim p_y}[D(G(z, y), y)] \tag{19}$$

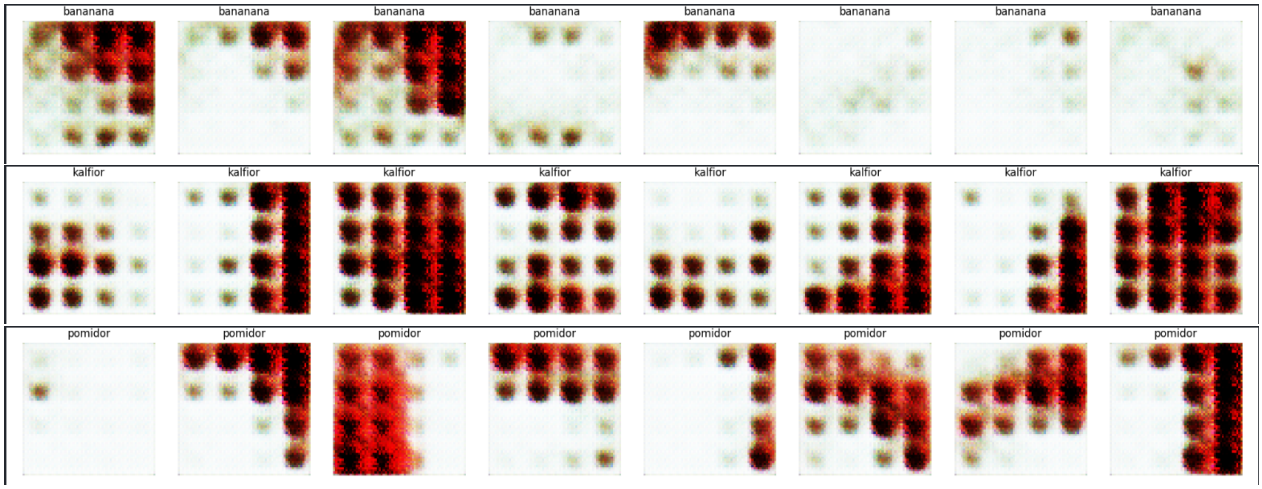
Zalety CWGAN:

- Zachowuje wszystkie zalety WGAN,
- Pozwala kontrolować, co jest generowane (np. wygeneruj pomidora),
- Stabilnie uczy się nawet przy małej ilości danych.

10 Wyniki WGAN (z weight clippingiem)

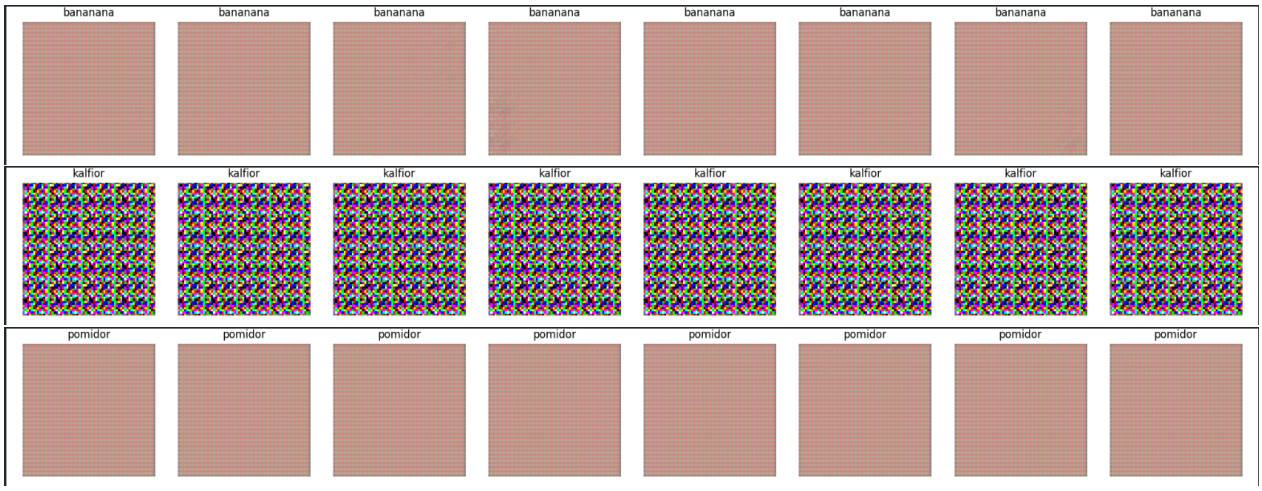
W kolejnym kroku zaimplementowałam klasycznego WGAN-a, który zamiast entropii krzyżowej używa odległości Wassersteina. Początkowo zastosowałam wersję z przycinaniem wag (weight clipping), aby spełnić warunek lipschitzowski. Jednak przycnianie wag, nie zrobiło nic oprócz pogoszenia moich juz i tak średnich według mnie wyników. Owoce i warzywa w tej wersji wyglądają, jakbym je rozgniotła tłuczkiem do ziemniaków.

Poniżej znajdują się przykładowe obrazy wygenerowane przez mojego pierwszego WGAN-a z przycinaniem wag i optymalizaotrem RMSProp:



Obrazy wygenerowane przez WGAN z clippingiem: banan, kalafior, pomidor

Poniżej znajdują się przykładowe obrazy wygenerowane przez mojego pierwszego WGAN-a z przycinaniem wag i optymalizaotrem Adam:

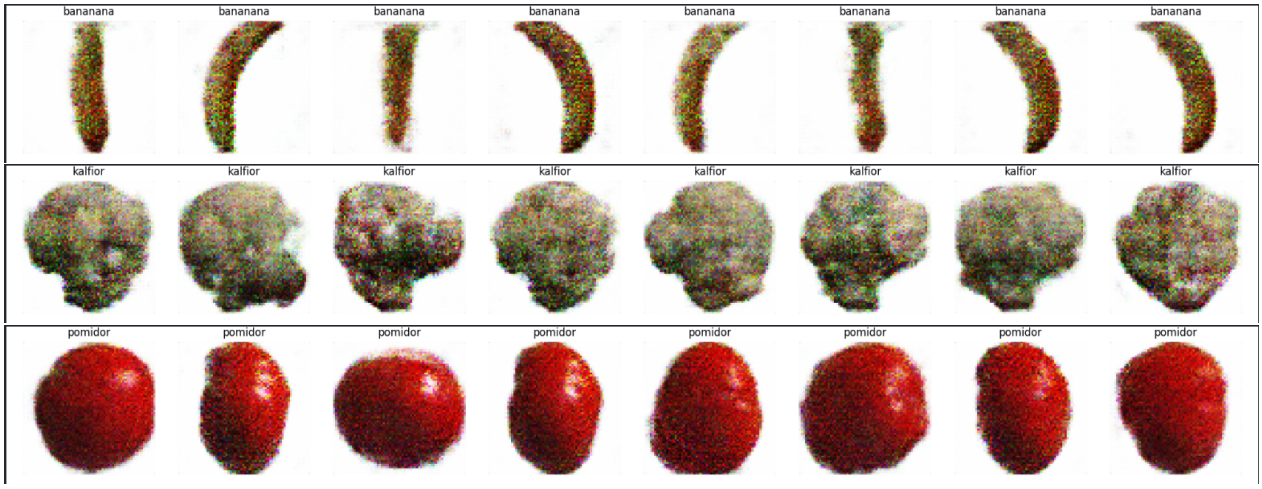


Obrazy wygenerowane przez WGAN z clippingiem: banan, kalafior, pomidor

11 Wyniki CWGAN-GP (z gradient penalty)

Ostateczna wersja mojego modelu to CWGAN z gradient penalty. Dzięki zastosowaniu GP zamiast weight clippingu. Model ten działał najlepiej ze wszystkich, jakie testowałam — generowane obrazy były bardziej realistyczne i bardziej odpowiadały przypisanym etykietom.

Poniżej znajdują się przykładowe wyniki końcowego modelu CWGAN-GP:



Obrazy wygenerowane przez CWGAN-GP: banan, kalafior, pomidor